REPUBLIC OF THE PHILIPPINES
POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
OFFICE OF THE VICE PRESIDENT FOR ACADEMIC AFFAIRS
COLLEGE OF COMPUTER AND INFORMATION SCIENCES

Girl Scout Cookie Night Delivery Dash

Case Study 1: Game Development Proposal in Introduction to Artificial Intelligence

By:

ALEJANDRO, Clarissa May C.
GARCIA, Phillip Dylan P.
SAMILLANO, John Eric B.
YBAÑEZ, Krisdel A.

BSCS 3-4

April, 2026

I. Introduction and its Background

Girl Scout Cookie Night Delivery Dash is an original two-player competitive delivery game set in a village at night. The main task of both players is to deliver cookies to three assigned houses and then exit the village as quickly as possible. The game combines route planning, memory, and risk management because players must move through a partially visible environment while avoiding hidden hazards.

The proposed digital game presents a simple but challenging delivery scenario. Since the setting takes place at night, players cannot see the whole map at once. Instead, they only see a limited area around their current position, as if they are using a flashlight. Because of this, players must move carefully, remember paths they have already explored, and decide which direction is safer or faster. Another important part of the game is the presence of hidden obstacles in the village. Some paths may contain puddles or broken roads that are not immediately visible to the player. These hazards make the game more exciting because they can delay movement and affect the player's performance. The limited vision and hidden traps add tension to the gameplay and make each round less predictable.

The game also includes a human-versus-AI setup, which makes it suitable for an Intro to AI project. The human player competes against an AI opponent that can make decisions based on pathfinding and route evaluation. The AI may use Depth-First Search (DFS) to explore possible paths toward the houses and the exit, while Heuristic-Driven Backtracking Search helps it return to previous decision points when a chosen path is inefficient, risky, or blocked by hidden obstacles. Through backtracking, the AI can reconsider other possible routes and continue searching for a better path to complete its deliveries. This allows the game to combine a simple delivery objective with basic artificial intelligence concepts.

Although both players use the same village map, they do not appear together on one screen. Each player has a separate view of the game, which increases suspense because they cannot directly monitor the opponent's location. This design keeps the competition active and creates pressure throughout the game.

Overall, the game offers a simple concept with interactive and strategic elements. It combines delivery mechanics, limited visibility, hidden obstacles, scoring, and AI-based decision-making in one experience. Because of these features, the game is not only competitive and engaging but also appropriate as a project that demonstrates how AI can be applied in gameplay.

II. Game Proper

A. Objective of the Game

The objective of Girl Scout Night Cookie Delivery is to deliver cookies to the three assigned houses and exit the village before the AI opponent. The player must complete the mission in the shortest possible time while also earning the highest possible score.

To achieve this, the player must choose efficient paths, avoid hidden obstacles, and use bonus items properly. The game rewards both speed and smart decision-making, since the winner is determined not only by finishing quickly but also by gaining more points during the game.

B. Discuss the Rule of the Game

Game Rules

- The game is played on a 10x10 village map.
- The game has two players: one human player and one AI player.
- Both players use the same village layout, but they are shown on different screens.
- The players cannot directly see each other while playing.
- Each player must deliver cookies to three houses in the village.
- After delivering to all three houses, the player must go to the exit point.
- A player cannot win unless all three houses have already received the cookies and exit the village.
- The game uses tile-based movement.
- A notification appears whenever a player successfully delivers cookies to a house. This notification adds pressure because both players know the opponent is making progress.

Obstacle Rules

- Some tiles contain hidden obstacles because the game takes place at night. These obstacles are not immediately visible to the player.
- If a player steps on a puddle, the player loses 150 points and resets their position 2 valid steps back along their current path
- If a player steps on a broken road, the player loses 200 points and resets their position 3 valid steps back along their current path

Bonus Item Rules

- The game includes bonus items that may appear randomly and can be received by the house after delivering the cookie.
- Boots protect the player from puddles.
- Rope protects the player from broken roads.

Visibility Rules

- The visible area changes every time the player moves.
- If the player changes position, the flashlight view also changes based on the new tile location.

Winning Rules

- The winner is the player who finishes all three deliveries and exits the village first.
- The game uses a score-and-time system to determine the winner.
- Players earn 50 points for every unique tile revealed.
- Players earn 800 points for every successful cookie delivery.
- Players lose 150 points for every puddle they step on.
- Players lose 200 points for every broken road they step on.
- The total score is computed using this formula:

$$TotalScore(S_{total}) = (w_u \cdot U) + (w_d \cdot D) - (w_p \cdot P + w_r \cdot R)$$

Where:

Variable	*Description*
U	***Unique Tiles Discovered:*** Only gives

	points the first time a tile is revealed.
D	***Successful Deliveries:*** Number of cookies delivered (max 3).
P	***Puddles Hit***: Number of puddle constraints encountered.
R	***Broken Roads Hit:*** Number of broken road constraints encountered.

After obtaining the total score, the final score is computed by dividing the total score by the total time (in seconds) to determine the final **Performance Index**.

$$PerformanceIndex(PI) = \frac{S_{total}}{T}$$

Variable	*Description*
T	***Time in Second:*** The overall duration the player took to deliver all cookies and find the exit.

- The player with the higher performance index is considered the better performer.
- This system makes both speed and accuracy important.
- A player may finish fast, but if they hit too many obstacles, their final score may be lower.
- The goal is to complete the mission with the highest final score.

Rules per Level

- Easy Level
  - The player can see a 3x3 area around the current position.
  - This level is easier because more tiles are visible.
  - It is more suitable for beginner players.
- Intermediate Level
  - The player can only see a 2x2 area around the current position.
  - This level is more challenging because the visible range is smaller.

    - The player must rely more on memory and careful movement.
- Hard Level
    - The player can only see a 1x1 area, or only the current tile.
    - This is the most difficult level.
    - The player has very limited map visibility.

Scoring Rules

- For every unique tile discovered, the player earns 50 points.
- The game also considers completion time.
- The goal is to get the highest score and the shortest time.
- Score and time may also be used to evaluate overall performance.

III. Implementation of AI Algorithm

A. AI Algorithm Implemented in the Game

The game utilizes a variety of algorithms to simulate intelligent behavior and ensure a fair, navigable environment. Listed below are the algorithms used in the implementation:

- **Heuristic-Driven Backtracking Search**

  This is the main algorithm that will be used by the AI to navigate the village. It is based on Depth-First Search (DFS) but is enhanced with heuristic guidance to help the AI choose more efficient paths. As the AI explores the map, it attempts to move toward its target locations, such as delivery houses or the exit. When it encounters hidden obstacles like puddles or broken roads, the algorithm triggers a backtracking process, allowing the AI to return to a previous safe position and try a different path. Through this approach, the AI is able to adapt to the partially visible environment, avoid repeating costly mistakes, and improve its overall route selection during gameplay.

```
ALGORITHM HeuristicBacktrack(current_pos, goals, memory)
  IF all goals are reached:
    RETURN True
```

```
// Greedy Step: Sort adjacent tiles by Manhattan Distance
neighbors = GetNeighbors(current_pos)
SortByManhattanDistance(neighbors, goals[0])

FOR EACH neighbor IN neighbors:
   IF neighbor IS NOT in memory.visited AND neighbor IS NOT in memory.pruned:
      MoveTo(neighbor)
      Add neighbor TO memory.visited

      IF neighbor is a Puddle OR Broken Road:
         TriggerPenalty(score)

         // Backtracking Step: Revert state
         RevertState(neighbor, penalty_steps)
         Add neighbor TO memory.pruned
         CONTINUE // Try next best neighbor

      IF HeuristicBacktrack(neighbor, goals, memory):
         RETURN True

RETURN False
```

- **Manhattan Distance Heuristic**

  To prevent the AI from moving randomly, a greedy heuristic is integrated into the backtracking search. The AI evaluates all possible adjacent moves and prioritizes the option that minimizes the estimated distance to its current target, such as a delivery house or the exit point. This allows the AI to make more directed and efficient decisions instead of exploring paths blindly.

  The Manhattan Distance formula is used as the heuristic measure because the game operates on a grid with four-directional movement (north, east, south, and west). This makes it more appropriate than other distance metrics, as it directly reflects the actual movement constraints of the environment.

$$d = |x_1 - x_2| + |y_1 - y_2|$$

This heuristic helps guide the AI toward its goal while still allowing the backtracking mechanism to handle unexpected obstacles and incorrect path choices.

```
ALGORITHM CalculateManhattanDistance(current_pos, target_pos)
  // current_pos and target_pos are (x, y) coordinates

  dx = ABS(current_pos.x - target_pos.x)
  dy = ABS(current_pos.y - target_pos.y)

  distance = dx + dy

  RETURN distance
```

- **Difficulty-Specific Algorithmic Variations**
  - **Easy (Stochastic Search with Memory Decay):** The AI uses a randomized heuristic where it has a probability of choosing a sub-optimal path. Additionally, it features Memory Decay, where its list of known pond locations is cleared periodically, causing it to potentially repeat past mistakes.
  - **Intermediate (Deterministic Greedy Search):** The AI follows a strict, non-random pathfinding logic. It utilizes Persistent Pruning, meaning once it discovers a constraint, that coordinate is permanently blocked in its memory for the remainder of the match.
  - **Hard (Lookahead DFS):** This version implements Predictive Pruning. Before moving to a tile, the AI "scans" the immediate surroundings. If a hazard is detected within its vision range, it avoids the tile entirely, simulating a highly skilled and cautious opponent.

```
ALGORITHM SelectNextMove(difficulty, neighbors, goals, memory)
  CASE difficulty:
    "EASY":
      IF random(0, 1) < 0.3: // 30% chance of a random move
        RETURN random_choice(neighbors)
      PerformMemoryDecay(memory, interval=10) // Forgets ponds
```

```
"INTERMEDIATE":
    // Pure Greedy choice with Persistent Memory
    RETURN neighbor with minimum ManhattanDistance(goals[0])

"HARD":
    // Predictive Pruning: Scan neighbors before moving
    FOR EACH n IN neighbors:
        IF ScanForHazards(n) == True:
            Add n TO memory.pruned // Prunes before stepping
    RETURN neighbor with minimum ManhattanDistance(goals[0])
```

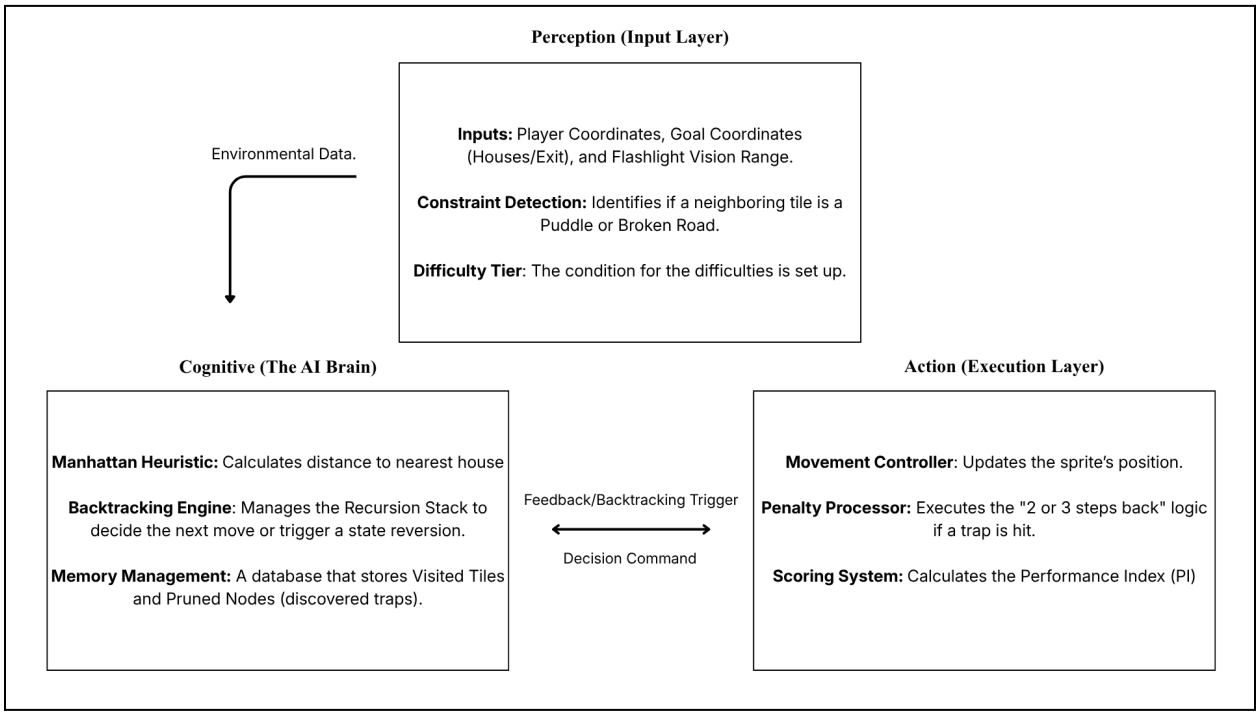
- **Breadth-First Search (BFS) for Map Validation**

  While the main gameplay relies on Depth-First Search (DFS) logic, Breadth-First Search (BFS) is employed during map generation to ensure playability. Because ponds and broken roads are randomly placed, there's a chance of producing an impossible map where a house becomes completely inaccessible. To prevent this, each newly generated map undergoes a BFS validation process that checks for at least one viable path connecting the starting point to all three houses and the exit. If no valid path is found, the system discards the map and regenerates it until a playable configuration is achieved.

```
ALGORITHM ValidateMap(start, houses, exit, grid)
    targets = houses + [exit]
    FOR EACH target IN targets:
        Queue = [start]
        Visited = {start}
        PathFound = False

        WHILE Queue is NOT empty:
            current = Queue.pop(0)
            IF current == target:
                PathFound = True
                BREAK
```

```
        FOR EACH neighbor OF current:
            IF neighbor IS reachable AND NOT in Visited:
                Add neighbor TO Visited
                Queue.append(neighbor)

    IF PathFound == False:
        RETURN False // Map is impossible

RETURN True // Map is valid
```

B. Software Architecture



System Architecture

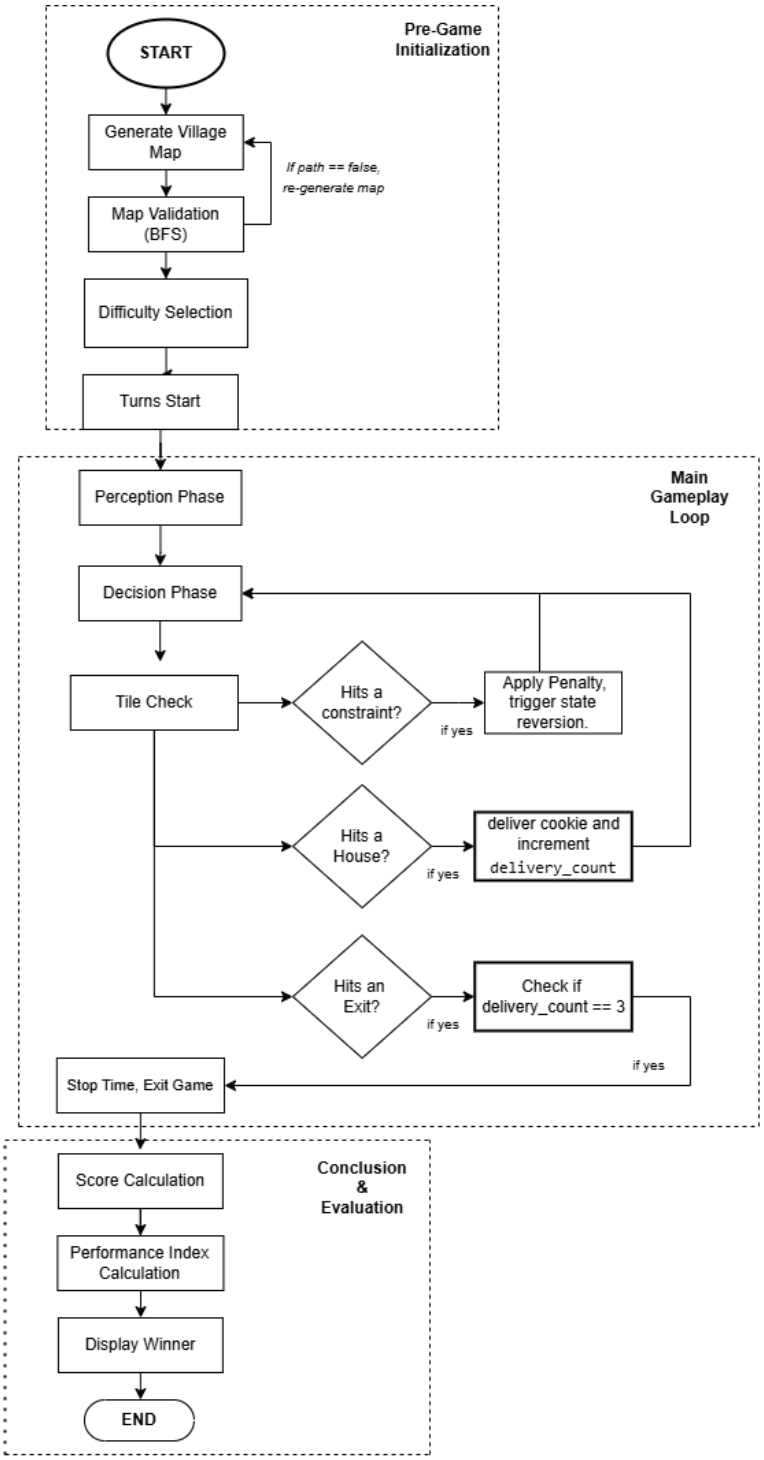
The software architecture for *Girl Scout Cookie Night Delivery Dash* follows a modular **Perception–Cognition–Action (PCA) loop**, which organizes how the AI receives information, makes decisions, and executes actions.

The process begins with the **Perception layer**, which gathers real-time data from the environment. This includes the AI's current position on the grid, the locations of assigned delivery targets, and the limited vision range based on the selected difficulty level.

The collected data is then passed to the **Cognition layer**, which serves as the AI's decision-making engine. In this stage, the AI evaluates possible adjacent moves using the Manhattan Distance heuristic to prioritize actions that bring it closer to its target. The backtracking mechanism works together with memory management to ensure that previously penalized or pruned coordinates are avoided, allowing the AI to make more informed and efficient decisions.

Once a decision is made, it is forwarded to the **Action layer**, where the movement controller updates the AI's position on the grid. If the AI encounters a constraint such as a puddle or broken road, the penalty processor triggers a state reversion, moving the AI back by a fixed number of steps. This event also sends feedback to the cognition layer so that the AI can update its memory and avoid repeating the same mistake.

Throughout this continuous loop, the **Scoring System** evaluates performance by computing a Performance Index (PI), defined as the total accumulated score divided by the elapsed time in seconds. This ensures that both efficiency and speed are considered in determining the final outcome of the game.



System Flowchart

The operational flow of *Girl Scout Cookie Night Delivery Dash* is divided into four main phases: initialization, the recursive gameplay loop, constraint handling, and performance evaluation.

The process begins with **pre-game initialization and map validation**. Before the match starts, the system generates a 10×10 village grid and places the delivery houses, hidden puddles, and broken roads. To ensure that the game remains fair and solvable, a Breadth-First Search (BFS) algorithm is executed. Unlike the AI's navigation logic, BFS is used here purely for validation. It checks whether all houses and the exit point are reachable. If no valid path exists, the map is discarded and a new one is generated.

Once the game begins, both the human player and the AI enter a **recursive navigation loop**. During each cycle, the system first performs perception by updating the visible tiles based on the selected difficulty level, such as a 3×3view for easy mode or a 1×1 view for hard mode. The AI then performs heuristic evaluation by analyzing its adjacent tiles using the Manhattan Distance formula to estimate how close each move brings it to its current target. It prioritizes the move with the lowest distance, resulting in a greedy and goal-oriented behavior.

During navigation, **constraint handling and backtracking** are triggered whenever a player or the AI steps on a hidden obstacle. The system applies a penalty, deducting points depending on the type of constraint encountered. At the same time, a state reversion occurs, where the character is moved back several valid steps along its recorded path. For the AI, this event also updates its memory by marking the coordinate as undesirable, allowing it to avoid repeating the same mistake in future decisions.

The loop continues until all delivery objectives are completed and the exit point is reached. At this stage, the system proceeds to **goal completion and performance evaluation**. The final ranking is determined by computing the Performance Index (PI), which is obtained by dividing the total accumulated score by the total time in seconds. This ensures that both efficiency and speed are considered in evaluating the player's overall performance.